

F1R3Skein

An Instrument, a Population, and a Theory of Skeins

F1R3FLY.io

4 September 2026

Abstract

F1R3Skein is the musical member of the F1R3Games suite, a collection of games built to develop and exercise collective intelligence on a F1R3FLY shard. It generates melody by reading the digit streams of mathematical constants through a pair of spigot algorithms, and then evolves that melody under the aesthetic selection of a community rather than the design of an individual. This note sets out what F1R3Skein is for, describes its architecture as it is being rebuilt against the cost-accounted rho backend, and argues for the representational commitment that distinguishes the new implementation from the old: a tune is a *term*, not a blob of notes. The consequences of that commitment—free lineage, canonical forms, syntactic breeding, and honest pricing of computation—are drawn out, and a Theory of skeins is given in the data definition sublanguage of flr3|@ng. The note closes with F1R3Skein’s relationship to the other four games, which is closer than a shared login page.

1 Intent

The F1R3Games suite exists under the motto *E Pluribus Sapiens*. Each game isolates one faculty that groups exercise badly or well, and puts it under conditions where the group’s performance becomes visible and recordable. F1R3Pix isolates coordination under communication constraints; F1R3Ink isolates affective convergence; F1R3SideChat isolates collaborative narrative. F1R3Skein, under *E Pluribus Cantus*, isolates **aesthetic judgement**.

Isolating aesthetic judgement requires separating it from composition, and this is the design’s first move. Generation in F1R3Skein is mathematical and deterministic: nobody composes the candidate melodies. Selection is collective and subjective: the community’s listening is the only force acting on the population. What emerges after some generations is a body of music that no individual designed and that no individual could have predicted, but which is nonetheless a faithful record of a particular group’s taste. The group’s ear is the fitness function, and the gallery is its fossil record.

Two design consequences follow immediately and are worth stating before any architecture.

First, **engagement must be the selection pressure**. A tune that is played, liked, tipped, snipped from, or bred from is a tune that some person actually attended to. Reproduction is biased toward such tunes. This closes the loop that makes the game a game rather than a generator.

Second, **the artifacts must be cheap to store and exact to reproduce**. A community’s evolutionary history is worthless if it is approximate. The whole architecture below is organised around making tunes reproducible from a few bytes, which is possible because the underlying generation is deterministic—and which becomes fragile the moment breeding is introduced. That fragility is the subject of Section 4.

2 The instrument

2.1 Spigots

A spigot algorithm emits the digits of a mathematical constant one at a time, in order, without computing the number to a fixed precision in advance. F1R3Skein’s engine supplies five such streams— π by Gosper’s unbounded linear-fractional-transformation spigot, e and $\ln 2$ by Rabinowitz–Wagon mixed-radix carry propagation, the golden ratio φ , and Liouville’s constant—each parameterised by an output base in the range $2 \dots 36$.

Two streams run side by side. One governs pitch, the other duration. Digits are mapped into a scale and a rhythmic vocabulary; the base is chosen to fit the vocabulary, so that a base-22 stream indexes a 22-pitch pentatonic spread across five octaves while a base-5 stream indexes five rhythmic values from a thirty-second note to a quarter. Interleaving the two produces a melodic phrase.

The crucial property is that a phrase has a **mathematical address**. The pair

$$((c_L, b_L), (c_R, b_R), [i, j])$$

of two configurations and an interval determines the phrase completely. Fresh spigots are created from the configurations, fast-forwarded to i , and $j - i$ digit pairs are collected. Nothing else is needed. A melody is therefore storable on chain as a handful of bytes rather than as audio, and two players who hold the same address hold, provably, the same melody.

2.2 Gestures

The player’s interface is gestural, and the gestures are named after what a weaver does with yarn. *Pulling* with the left or right hand advances the corresponding stream, faster or slower with the speed of the pull. *Twisting*—one hand crossing over the other—swaps which stream governs pitch and which governs duration. *Clapping* stitches the two ribbons together and begins playback; *unclapping* separates them and stops. A *scissors* motion snips the stitched section into a named tune, which drops into a tray.

The two ribbons are rendered as receding rows of coloured patches, one colour per digit value, with stitch lines spanning between them during playback. The engine is driven either by a Leap Motion controller, by an Apple Vision Pro over a local socket carrying newline-delimited JSON, or by keyboard simulation for players without hardware. All three speak the same small event vocabulary, which matters for Section 3.

3 Tunes and performances

F1R3Skein’s gallery is subtler than the galleries of the other four games, because it holds two kinds of artifact rather than one.

Definition 1 (Tune). A *tune* is a snipped, named, evolvable object: the thing a player deliberately preserved and the thing that participates in reproduction.

Definition 2 (Performance). A *performance* is the recorded trace of a session at the instrument: an ordered, timestamped sequence of gesture and parameter events together with the opening configuration of each stream. A performance may yield zero, one, or several tunes—those the performer chose to snip—but it is a separate artifact with its own identity.

The distinction earns its keep in three ways. A performance is watchable and listenable as a *piece* in its own right, with a shape and an arc that its harvested fragments do not have. A performance carries the provenance of its tunes: hearing a fragment in the context where it was found is often the best argument for it. And a performance is engageable and tippable

independently, which means a player who is a fine improviser but an indifferent curator can be rewarded for what they are good at.

Because the streams are deterministic, a performance need store no audio either. The trace replays exactly. This is the same economy that makes tunes cheap, applied one level up.

4 The representation problem

4.1 Where the old implementation breaks

The address representation of Section 2 is exact and compact, and it survives every operation the instrument performs—pulling, twisting, snipping. It does *not* survive breeding.

The existing breeding surface crosses two tunes to produce four children. The first takes the mother’s pitches with the father’s durations; the second the converse; the third folds pitch against pitch through base 5 to yield a duration; the fourth lifts duration against duration through base 22 to yield a pitch, with a coin flip choosing the direction in each of the last two. Unequal lengths are equalised by inserting rests at random positions.

Every one of these operations destroys the address. The child of two addressed tunes is not, in general, any interval of any pair of spigots. The implementation therefore falls back at the first generation to storing an explicit vector of pitch-duration pairs, with a **parents** pointer bolted on to recover lineage after the fact.

This is a representational fault line, and everything downstream inherits it. Lineage becomes a side table rather than a property of the object, so it can be wrong. Two tunes that are provably the same tune are stored twice, because equality is only decidable on realised note vectors. And a tune and a recipe for a tune become different types travelling different storage paths, which doubles the surface area of every gallery, every query, and every migration.

4.2 The commitment

A tune is a term. Its leaves are spigot addresses; its internal nodes are the snip, twist, pad, crossover and mutation operators. Realisation—turning a term into notes—is a rewrite.

Four things follow, and together they are the argument for the whole approach.

Lineage is free. A bred tune’s ancestry is not recorded next to it; it *is* its own syntax. The **parents** pointer disappears, and with it the class of bugs in which lineage and content disagree. The gallery’s lineage view becomes a rendering of a term tree rather than a walk over a join table.

Canonical forms are available. Equations over the operators induce normal forms, so the gallery de-duplicates by normalisation rather than by hashing audio. Section 5 shows that the equations are not decorative: one family of them collapses a whole class of duplicate entries that the current implementation would store separately.

Breeding becomes syntactic. Crossover is an operation on terms, which is exactly the capability the data definition sublanguage of `f1r3|@ng` exists to provide. Rewriting, matching, and search over tunes become available as language features rather than as bespoke code.

Computation is priced honestly. Cost accounting attaches at the realisation rewrite. Emitting the n -th digit of π costs phlo , so a deep address costs more than a shallow one. An evolutionary economy needs a resource floor, and this is a real one rather than a stipulated fee schedule.

4.3 Building it twice, on purpose

The Theory syntax is not yet integrated into the backend. The response is not to wait, and not to proceed without it, but to build the representation twice behind a single interface.

Version `skein/v0` is a hand-written algebraic data type in the engine crate, with a hand-written normaliser. Version `skein/v1` is the elaborated Theory. The `v0` type is deliberately *isomorphic* to the intended Theory signature—same constructor names, same arities, same argument names—so that migration is a re-parse and a re-target rather than a re-modelling. Every stored tune carries its representation version in an envelope; nothing above the engine branches on it; galleries, engagement, invitations, performances and token rails see only a stable public interface.

The Theory itself is written at the *start* of this process rather than at the end, and the `v0` type is property-tested against it. Writing it early is what makes the migration cheap. Writing it late would mean discovering, at migration time, that `v0` had committed to something the Theory cannot say. The migration’s acceptance test is then a single equation—for every tune in the gallery, realising the `v0` term and realising its migrated `v1` term must give the same notes.

5 A Theory of skeins

The Theory is presented in the module syntax frozen in August 2026: theories are declared with `Theory  $T(p:P)$` , terms are given in judgement form `Label . ctx |- syntax : Cat`, categories are declared in a `Types` block and made visible in an `Exports` block, and `\/` is a pushout over shared parameters. What follows is a summary; the full module accompanies this note.

5.1 Melody is a monoid

Concatenation of tunes is associative, has a unit—the rest—and is *not* commutative. It is therefore not defined afresh. It is the plain monoid of the universal-algebra tower, with its carrier renamed:

```
Theory Melody(m: u.Monoid) {
  m
  Exports {
    Elem => Tune;
  }
  Replacements {
    One  => Rest . |- "-" : Tune;
    Mult => Seq . ts:List(Tune) |- "[" ts.*sep(";") "]" : Tune;
  }
}
```

This is worth pausing on, because it places `F1R3Skein` and `rholang` as siblings. `Rholang`’s parallel composition takes the *commutative* monoid and a hash bag, because $P \mid Q \equiv Q \mid P$ and the bag makes that structural. `Melody` takes the *plain* monoid and a list, because a tune played forwards is not the tune played backwards. Two products of the same tower, distinguished by one equation. Everything proved about monoids comes with each of them.

The same move gives counting for free: positions and lengths form $(\mathbb{N}, +, 0)$, a commutative monoid, and advancing a stream is a monoid *action* of it—which is precisely what the engine’s `advance` method implements, and which the Theory records as two equations.

5.2 Crossover is a rectangular band

Write $m \bowtie f$ for the child taking m ’s pitches and f ’s durations. Then:

Proposition 1. $\bowtie$ is idempotent and associative and satisfies $a \bowtie b \bowtie c = a \bowtie c$; that is, $(\text{Tune}, \bowtie)$ is a rectangular band.

Proof. $t \bowtie t$ takes t 's pitches and t 's durations, so equals t . For the absorption law, $(a \bowtie b) \bowtie c$ takes the pitches of $a \bowtie b$ —which are a 's—and the durations of c ; and $a \bowtie (b \bowtie c)$ takes a 's pitches and the durations of $b \bowtie c$, which are c 's. Both equal $a \bowtie c$, giving associativity and absorption together. $\square$

Two practical consequences. The second child of the existing breeding function, which takes the father's pitches with the mother's durations, is simply $f \bowtie m$, so it needs no constructor of its own; the constructor count halves. And the band laws are a *testable property of the existing JavaScript*. Differential testing the engine against the current `breed()` is a scheduled task, and if the laws fail, the JavaScript has a bug.

5.3 The awkward case: stochastic operators

The third and fourth children involve a coin flip. A term whose realisation depends on an unseen random draw does not faithfully describe its own realisation, and the claim that lineage is syntax collapses. The Theory therefore gives each of these operators an explicit seed argument, restoring functionality:

```
XFold . a:Tune, b:Tune, seed:Nat |- "fold" "(" a "," b "," seed ")" : Tune;
XLift . a:Tune, b:Tune, seed:Nat |- "lift" "(" a "," b "," seed ")" : Tune;
```

The alternative—modelling them as relations rather than functions—buys cleaner algebra at the price of non-deterministic realisation. This remains an open decision, and it is the one decision that blocks implementation of the term type.

5.4 A trap the elaborator catches

The counting tower and the melody tower must descend from *independent* base instances. Passing one carrier to both would identify the category of positions with the category of tunes, asserting that an index into a digit stream and a piece of music are the same kind of thing. Under the older grammar, where the exports block did double duty as declaration, this modelling error was invisible. Under the frozen syntax it is a join collision and a diagnostic. The elaborator holds the design to its own claims about sharing, which is the strongest practical argument for writing the Theory before the code rather than after it.

One genuine gap is exposed in the other direction. The frozen grammar supplies a hash bag but no sequence sort, and melody needs order. `List(Tune)` above is a request against the data definition sublanguage, not a feature of it today. A second, subtler issue arrives with the same replacement: a binary operation replaced by a list-valued one leaves the inherited binary associativity equations written over a now list-valued constructor, where the list has already made them structural. That is the open arity-drift question, and melody is where it bites.

6 Architecture

6.1 Backend

The implementation targets `f1r3node-rust` on the cost-accounted rho feature branch. Three capabilities of that branch are load-bearing.

Cost accounting supplies the metering that prices realisation, together with the deterministic reduction and byte-accounting machinery that makes those prices reproducible across validators.

A *capability registry* exposed as a system contract provides content-addressed, bounded-use, revocable capability handles. An invitation to the platform is naturally such a handle: inviting n friends registers a handle with a use bound of n , redemption is an invocation, withdrawal is a revocation, and registration is idempotent on the tuple, so re-issuing an invitation is safe. Derivative rights on a tune—the right to breed from it, sample it, or include it in a performance—take the same shape.

A *conserving exchange* implemented as a persistent join over two carrier channels provides the tipping and cross-game conversion rail. Its invariant is per-channel conservation: the join consumes exactly one datum from each carrier and emits exactly one on each. An engagement economy that settles through it cannot mint.

Where clauses are live on the branch, guarding both receives and match arms. They are the natural home for selection predicates, and the graded form is the natural home for fitness as a grade rather than as a number computed off to one side.

6.2 Service and clients

A single service mediates between the browser and the node, laid out on the pattern already proven by the wallets-and-agents service in the F1R3FLY estate: an HTTP surface, a domain layer, an on-chain model layer, and templated rholang. State lives in tree hash maps behind a versioned environment contract deployed through the registry’s signed-insert path, with each domain exposing a method-dispatch contract and keeping a `latest` alias alongside its version history.

Identity is web3 and shard-native. A player generates a secp256k1 key pair in the browser; the public key yields the address; the private key never leaves the client and is the sole credential. Sign-on is a signed challenge. There are no passwords and no email. One key pair is one identity across all five games, which is what makes cross-game analytics and a shared engagement ledger possible at all.

The realisation engine is compiled once, in Rust, and delivered to the browser as WebAssembly, so that a tune realised on a player’s machine is bit-identical to the same tune realised on the service. A re-implementation in JavaScript would eventually diverge, and divergence here is not a rendering artifact—two players would breed different children from the same parents, and the gallery’s lineage would become a fiction.

7 Relationship to the other F1R3Games

All five games share one entry point, one identity, one invitation mechanism, one gallery pattern, and one engagement ledger. Beyond that shared platform, F1R3Skein has specific relationships with each.

F1R3Beat is the closest. Beat evolves rhythmic grids where Skein evolves melody, and the two are best understood as one evolutionary population viewed through two instruments. A Skein tune projects to a Beat pattern through the same base-5 duration map the breeding engine already uses; the engagement ledger is shared; a tune bred under Skein’s selection pressure can be selected under Beat’s. The cross-feed is the single most valuable integration in the suite, because it doubles the selection pressure acting on any given lineage.

F1R3Pix contributes the invitation and contact-collection surface, generalised in the rebuild to a shared component used by every game—with the standing exception of the Skein browser, which does not use it. Pix and Skein otherwise exercise different faculties and share only the platform.

F1R3Ink supplies an affective vocabulary. Ink’s business is finding shared language for shared feeling through colour; that vocabulary is a natural tagging scheme for tunes, and a longitudinal record of a community’s affect is a natural covariate for a longitudinal record of its taste. The integration is optional and downstream.

F1R3SideChat is where performances go to be talked about. A performance is a narrative artifact as much as a musical one, and SideChat’s tiered participation—free comment, paid character control, expensive dialogue with the author—maps onto a performance as readily as onto a story. The tipping rail is shared.

A note of housekeeping belongs here. The portal’s current game listing describes F1R3Skein as an exercise in collective *narrative* intelligence, with copy about weaving threads of story. That description belongs to F1R3SideChat, and was attached to Skein during an early build session in which placeholder narrative contracts were written under Skein’s name. The rebuild corrects the copy and re-homes those contracts.

8 Open questions

1. **Stochastic operators.** Explicit seeds, or relational modelling? This blocks the term type and is the first decision to make.
2. **Performance storage.** Full trace on chain, or an on-chain digest with the body in the file system layer? Consistency with the mail-on-chain work argues for the former.
3. **The sequence sort.** List is a request against the data definition sublanguage. Until it exists, melody’s concatenation cannot be written as intended.
4. **Arity drift through replacements.** Whether replacements must be arity-preserving, or whether a replacement can declare that inherited equations have become structural, is undecided, and melody is a case in point.
5. **The fitness function.** Its weights are a governance object rather than an implementation detail. Whether they are set by deploy or by the voting application should be settled before the token economy goes live.
6. **Resistance to gaming.** Fitness driven by engagement is an attack surface as soon as tokens are attached. Sybil resistance on engagement counts is the sharp version of the problem.

Accompanying material

- *F1R3Skein — Phased Development Plan*: the audit of the current repositories and backend branch, the phase structure, effort estimates, risks, and the decisions required before implementation begins.
- `Skein.module`: the Theory of skeins in full, in the frozen data definition syntax.